2026

1º Trabalho

Modelação e Programação

REVISÕES, CLASSES E OBJECTOS

DATA DE ENTREGA: 15/03/2026
Data-Limite da Discussão: 29/03/2026

Avaliação

Avaliação: Este trabalho é para ser **realizado individualmente.** A execução deste trabalho é obrigatória. Cada questão deste trabalho está associada a uma pontuação específica, indicada na própria questão. Para alcançar tal pontuação, a aplicação desenvolvida pelo aluno deverá satisfazer integralmente todas as exigências propostas no enunciado, e o aluno deverá ser capaz de apresentar e justificar o trabalho efetuado na respetiva questão. Na eventualidade de não se cumprir esta condição, a pontuação atribuída nessa questão será de zero. A apresentação do trabalho terá de ocorrer nas aulas práticas da unidade curricular, até à data-limite divulgada na capa deste trabalho. A falta de participação na discussão resultará na reprovação à unidade curricular.

Uso da Inteligência Artificial

A integração da Inteligência Artificial na geração de código é um marco na engenharia de software. Contudo, em determinados momentos desta unidade curricular, o uso de ferramentas de geração automática (incluindo o *autocomplete*) será **explicitamente vedado**. Esta restrição prende-se exclusivamente com objetivos pedagógicos: assegurar que cada estudante desenvolve a capacidade de resolver problemas de raiz, de forma autónoma e sem auxílio externo. Consolidar estes fundamentos é vital para uma compreensão profunda da matéria. A utilização de IA, nestas partes do trabalho, **implica a invalidação de todo trabalho (nota zero).**

Por outro lado, haverá componentes do trabalho onde a utilização de IA é **integrante ou obrigatória**, de forma a preparar os alunos para as práticas modernas de desenvolvimento. O regime de utilização (permitido ou proibido) será indicado claramente em cada enunciado."

Relatório:

A entrega de um relatório **é obrigatória** para este trabalho. Para a sua elaboração, **é permitida** a utilização de ferramentas de Inteligência Artificial. Adicionalmente, o código fonte deve estar devidamente comentado seguindo as normas semânticas do Javadoc, sendo necessária a entrega dos *outputs* gerados por esta ferramenta em conjunto com o código.

Nota acerca da metodologia de execução do trabalho: Encoraja-se enfaticamente que, durante a elaboração deste trabalho, os estudantes procedam à pesquisa na internet sobre o funcionamento dos algoritmos e operações matemáticas solicitadas, <u>evitando a tentação de utilizar (ou procurar compreender) códigos previamente desenvolvidos</u>, e que **comecem o seu desenvolvimento a partir de uma página vazia.**

O aluno deverá colocar as várias resoluções relativas a este trabalho no package tp1.

Parte A – Revisões

Coloque as alíneas deste grupo no package tp1.Revisoes

A1 (2v) *(Sem IA e sem autocomplete*

Desenvolva o programa **CatalogoLivros** que, no seu método **main**, execute o seguinte:

a) Deve solicitar ao utilizador o número de livros que pretende inserir no catálogo.
b) Para cada livro, deve solicitar o título e o ano de publicação.
c) Criar estruturas de dados (e.g. arrays) para armazenar os títulos e os anos de publicação.
d) Ordenar os livros por ano de publicação por ordem crescente (implementar e usar, por exemplo, o método bubblesort).
e) Calcular e exibir na consola as seguintes informações:

- O ano de publicação mais antigo
- O ano de publicação mais recente
- O ano médio de publicação (valor inteiro)
- Quantos livros foram publicados antes do ano 2000
- Lista de todos os livros publicados na década de 2010 (entre 2010 e 2019, inclusive)

Nota: Em caso de introdução de valores inválidos (por exemplo, valores não numéricos, anos negativos ou anos futuros), deve ser apresentada uma mensagem de erro e solicitado novamente ao utilizador que introduza os dados corretos.

A2 (2v) *(Sem IA e sem autocomplete)*

Crie o programa **GestaoHorarios** que, no seu método **main**, deve solicitar ao utilizador o número de disciplinas que pretende gerir (máximo 10 disciplinas). Para cada disciplina, o programa deve solicitar:

- Nome da disciplina
- Dia da semana (1=Segunda, 2=Terça, 3=Quarta, 4=Quinta, 5=Sexta)
- Hora de início (formato 24h, por exemplo: 9 para 9h00, 14 para 14h00)
- Duração em horas (1, 2 ou 3 horas)

Após a introdução de todas as disciplinas, o programa deve oferecer ao utilizador as seguintes operações:

a) **Visualizar horário completo**: Exibir todas as disciplinas organizadas por dia da semana e ordenadas por hora de início dentro de cada dia;
b) **Detetar conflitos:** Verificar se existem disciplinas que se sobrepõem no mesmo dia (horários conflituosos) e listar essas disciplinas;
c) **Estatísticas semanais:** Calcular e exibir:
   - O dia da semana com mais horas de aulas
   - O dia da semana com menos horas de aulas
   - A média de horas de aulas por dia

- O número total de horas de aulas na semana

d) **Pesquisar disciplina:** Permitir ao utilizador introduzir o nome de uma disciplina e exibir toda a informação sobre essa disciplina (dia, hora, duração).

Nota: Para cada uma dessas operações, o programa deve exibir o resultado na consola de forma clara e organizada. Em caso de introdução de valores inválidos (por exemplo, dias fora do intervalo 1-5, horas fora do intervalo 8-20, durações inválidas, nomes vazios), deve ser apresentada uma mensagem de erro e solicitado novamente ao utilizador que introduza os dados corretos. Para esta questão, é obrigatório utilizar arrays para armazenar as informações das disciplinas (pode usar arrays bidimensionais).

A3 (2v) *(Sem IA e sem autocomplete)*

Este trabalho visa exercitar a recursividade, a manipulação de arrays bidimensionais e a formatação de saída em Java. Pretende-se desenvolver um programa que, em Java, gere na consola uma representação fractal utilizando caracteres ASCII. O fractal escolhido para este exercício é o Triângulo de Sierpinski. As especificações são as seguintes:

Estrutura de Dados:

Utilize uma matriz bidimensional de caracteres (por exemplo, `char[][]canvas`) para armazenar a forma do fractal. Inicialmente, cada posição da matriz deve ser preenchida com um caractere de espaço (' ').

Geração do Fractal:

Crie a função recursiva

```
private static void drawSierpinski(char[][] canvas, int row, int col, int size)
```

para desenhar o Triângulo de Sierpinski, tendo em conta o seguinte:

- Quando o tamanho do triângulo é 1, o caractere "*" é colocado na posição do array correspondente às variáveis `row` e `col`, que no início da recursividade são definidas como zero na função principal.
- No passo recursivo, a função desenha três triângulos menores (topo, canto inferior esquerdo e canto inferior direito) nas posições adequadas dentro da matriz (usando `row,col,size`).
- O tamanho do fractal é controlado através da constante inteira `size` (potência de 2), que representa tanto a altura do do triângulo quanto a metade do comprimento total da base (ou seja, a matriz terá uma dimensão de `size` por `(2* size)`.

Saída:

*	* * *	* * * * * * * * *	* *
Size = 1	2	4	8

A4 (2v) *(Sem autocomplete. IA apenas para consulta de APIs)*

Crie o programa **AsciiArtGallery** que, no seu método **main**, funcione como um gestor de arte ASCII armazenada em ficheiros de texto. O programa deve ler todos os ficheiros .txt presentes numa pasta específica (por exemplo, ascii_art/) e oferecer ao utilizador as seguintes operações através de um menu interativo:

a) **Listar ficheiros:** Exibir na consola uma lista numerada de todos os ficheiros .txt encontrados na pasta, mostrando apenas o nome de cada ficheiro (sem a extensão);
b) **Visualizar arte:** Solicitar ao utilizador o número correspondente a um ficheiro da lista e exibir todo o conteúdo (arte ASCII) desse ficheiro na consola;
c) **Copiar para clipboard:** Solicitar ao utilizador o número correspondente a um ficheiro e copiar todo o seu conteúdo para a área de transferência (clipboard) do sistema, confirmando a operação com uma mensagem;
d) **Apagar ficheiro:** Solicitar ao utilizador o número correspondente a um ficheiro, pedir confirmação (S/N), e apagar o ficheiro caso confirmado. Deve atualizar automaticamente a lista de ficheiros disponíveis;
e) **Pesquisar por palavra:** Solicitar uma palavra-chave ao utilizador e listar todos os ficheiros cujo conteúdo contenha essa palavra (útil para encontrar arte ASCII específica, como "gato", "robot", etc.);
f) **Sair:** Terminar o programa.

O programa deve validar todas as entradas do utilizador (números de ficheiro válidos, confirmações S/N, etc.). Em caso de erros (pasta não encontrada, ficheiro não existe, permissões insuficientes para apagar), deve apresentar mensagens de erro claras e manter o programa em execução. Para esta questão, é obrigatório utilizar arrays para gerir a lista de ficheiros e operações de leitura/escrita de ficheiros para aceder ao conteúdo da arte ASCII.

Nota: Para testar o programa, deve criar previamente uma pasta ascii_art/ com pelo menos 5 ficheiros .txt contendo diferentes exemplos de arte ASCII.

A5 (2v) *(Sem autocomplete. IA apenas para consulta de APIs)*

Crie o programa **AnalisadorTexto** que, no seu método **main**, analise a frequência de letras num ficheiro de texto e gere um histograma exportável para Excel. O programa deve executar as seguintes operações:

a) Solicitar ao utilizador o nome/caminho do ficheiro de texto a analisar (por exemplo, texto.txt);
b) Ler o conteúdo completo do ficheiro e contar a frequência de cada letra do alfabeto (A-Z), ignorando distinção entre maiúsculas e minúsculas. Caracteres que não sejam letras (números, pontuação, espaços) devem ser ignorados;
c) Calcular e exibir na consola as seguintes estatísticas:
   - O número total de letras analisadas
   - A letra mais frequente e o seu número de ocorrências
   - A letra menos frequente (excluindo letras com 0 ocorrências) e o seu número de ocorrências
   - A percentagem de vogais (A, E, I, O, U) em relação ao total de letras
d) Apresentar no ecrã um histograma de todas as letras do alfabeto, mostrando a frequência através de barras horizontais compostas por asteriscos (*). Cada asterisco deve representar aproximadamente 2% da frequência relativa;

Exemplo de formato do histograma:

```
A: ********** (145 ocorrências, 8.5%)
B: *** (52 ocorrências, 3.1%)
C: ******* (98 ocorrências, 5.8%)
...
Z: * (12 ocorrências, 0.7%)
```

e) Gravar os dados num ficheiro CSV (comma-separated values) chamado histograma.csv com o seguinte formato:

```
Letra,Frequencia,Percentagem
A,145,8.5
B,52,3.1
C,98,5.8
...
Z,12,0.7
```

Este ficheiro pode ser aberto diretamente no Microsoft Excel ou LibreOffice Calc, permitindo ao utilizador criar gráficos personalizados.

Nota: Em caso de erros (ficheiro não encontrado, impossibilidade de criar o ficheiro CSV, ficheiro vazio ou sem letras), deve ser apresentada uma mensagem de erro clara e o programa deve terminar de forma controlada. É obrigatório utilizar **arrays** para armazenar as contagens de frequência das letras e operações de **leitura/escrita de ficheiros**. O formato CSV deve usar vírgula como separador e ponto como separador decimal.

Parte B – A Biblioteca

Coloque as classes deste cenário no package tp1.library

(Sem autocomplete. IA apenas para consulta de APIs)

B1 (2.5v) Considere o seguinte enunciado, criado para um trabalho de programação em Java, com o objetivo de promover a aprendizagem da programação orientada a objetos. Pretende-se desenvolver uma aplicação informática para gerir uma biblioteca. Nesta aplicação, deverá ser modelado um conjunto de entidades (classes) que representem os diferentes tipos de obras disponíveis, os seus autores, e também os funcionários e utilizadores da biblioteca (bibliotecários, assistentes administrativos, sócios, etc.). Cada uma destas entidades terá métodos e atributos que deverão ser definidos, discutidos com o professor, implementados e posteriormente defendidos pelo aluno na discussão. Devem ser criadas pelo menos duas classes (e no máximo quatro) por cada entidade (por exemplo, `Livro`, `Revista`, `Autor`, `Bibliotecario`, `Socio`, `Emprestimo`, etc.). Cabe ao aluno refletir, desenhar em UML, debater com o docente e implementar as classes e as associações entre as várias entidades.

Para cada entidade criada, deverá incluir o seguinte:

- Construtor(es);
- Métodos acessores, tendo em conta que certos atributos não podem ser alterados depois de instanciados. Por exemplo, no caso de um sócio, não se deve permitir a alteração do nome, do Número de Sócio, do BI, do Número de Contribuinte (NC) e da data de nascimento após a sua criação; no caso de um livro, o ISBN e o título não devem ser alteráveis após a instanciação;
- toString;
- print;
- equals.

Nota: Antes de começar a programar, comece por desenhar o UML das classes utilizando uma das muitas ferramentas disponíveis para o efeito como, por exemplo:

- https://online.visual-paradigm.com/diagrams/solutions/free-class-diagram-tool/
- Usando o package TikZ-UML do latex (se quiser desenhar o diagrama programaticamente).
- Editor UML do IDE que o aluno esteja a utilizar (e.g. o uml-designer do eclipse).

Implemente os métodos que executam as funcionalidades supra descritas e, no método **main** da classe **TesteBiblioteca**, instancie vários objetos de cada uma das classes e mostre o output (print) na consola de cada entidade. Exemplo:

TesteBiblioteca.java

```java
public class TesteBiblioteca {
    public static void main(String[] args) {

        // Instanciar obras
        Livro livro1 = new Livro("O Senhor dos Aneis", "978-0261102354", 1954, 1178);
        Revista revista1 = new Revista("National Geographic", "0027-9358", "Março 2024", 80);

        // Instanciar autores
        Autor autor1 = new Autor("J.R.R. Tolkien", "Britânico", 1892);

        // Instanciar funcionários
        Bibliotecario bib1 = new Bibliotecario("Ana", "Referência e Periódicos");
        Administrativo adm1 = new Administrativo("Carlos", "Recursos Humanos");

        // Instanciar sócios
        Socio socio1 = new Socio("Beatriz", 1042);

        // Apresentar na consola as instâncias criadas
        System.out.println("=== Obras ===");
        System.out.println(livro1);
        System.out.println(revista1);

        System.out.println("\n=== Autores ===");
        System.out.println(autor1);

        System.out.println("\n=== Funcionários ===");
        System.out.println(bib1);
        System.out.println(adm1);

        System.out.println("\n=== Sócios ===");
        System.out.println(socio1);
    }
}
```

Consola:

```
=== Obras ===
Livro {titulo=O Senhor dos Aneis, isbn=978-0261102354, anoPublicacao=1954, numPaginas=1178}
Revista {titulo=National Geographic, issn=0027-9358, edicao=Março 2024, numPaginas=80}

=== Autores ===
Autor {nome=J.R.R. Tolkien, nacionalidade=Britânico, anoNascimento=1892}

=== Funcionários ===
Bibliotecario {nome=Ana, secção=Referência e Periódicos}
Administrativo {nome=Carlos, departamento=Recursos Humanos}

=== Sócios ===
Socio {nome=Beatriz, numSocio=1042, ativo=true}
```

Parte C – A Classe Biblioteca

(Sem autocomplete. IA apenas para consulta de APIs)

C1 (2.5v) Adicione ao projeto a classe **Biblioteca**. Esta classe deverá conter um conjunto de obras, funcionários, sócios e outras entidades desenvolvidas na alínea anterior. Atualize o diagrama UML, tendo em conta o seguinte:

A Biblioteca possui um nome, uma morada e a capacidade de adicionar novas obras, funcionários, sócios e outras entidades relevantes definidas na alínea B. Deve igualmente permitir visualizar na consola e obter todas as obras e sócios existentes na biblioteca. Além disso, deve facultar a referência para um sócio, desde que seja fornecido o respetivo Número de Sócio ou BI. Por fim, deve oferecer uma estrutura de dados onde constem todas as obras que se encontram atualmente requisitadas por um determinado sócio.

Nota: O aluno tem igualmente a possibilidade de adicionar outras funcionalidades, como por exemplo a gestão de datas de devolução, penalizações por atraso, ou pesquisa de obras por autor. Tanto a classificação desta alínea, como a da alínea B (majorada em 5 valores), está parcialmente dependente da originalidade e qualidade do programa apresentado na discussão.

Parte D – Programação com Agentes para Geração Automática de Código

(AI obrigatório)

D1 (5V) A geração automática de código e a programação baseada em agentes representam uma grande mudança na forma como a engenharia de software é abordada e praticada. Em vez de depender exclusivamente da programação manual e dos fluxos de desenvolvimento tradicionais, estas tecnologias permitem que os sistemas apoiem os programadores através da geração de código, da sugestão de soluções e até da realização de ações autónomas dentro do processo de desenvolvimento.

Esta evolução está a transformar a engenharia de software num esforço mais colaborativo entre humanos e agentes inteligentes, melhorando a produtividade, acelerando os ciclos de desenvolvimento e reduzindo tarefas repetitivas. À medida que os sistemas baseados em agentes se tornam mais capazes, estão também a redefinir a forma como o software é concebido, testado e mantido, marcando um passo significativo em direção a ambientes de desenvolvimento mais adaptativos e inteligentes.

A tua Missão

Nesta secção, a utilização de IA é obrigatória. Podes utilizar uma variedade de plugins, ferramentas e modelos de linguagem de grande escala para apoiar o teu trabalho, incluindo Claude Code (https://claude.com/product/claude-code), ferramentas de automação do GitHub como o Copilot (https://github.com/copilot), Codex da OpenAI (https://openai.com/pt-PT/codex/), Kiro (https://kiro.dev/) ou o Google Antigravity (https://antigravity.google/), consoante o que melhor se adequar ao teu fluxo de trabalho.

Esta tarefa inicia-se, antes de qualquer implementação, com a elaboração de um enunciado completo para uma aplicação à escolha do aluno. O objetivo principal é que cada estudante conceba e descreva, de forma detalhada, uma aplicação que venha posteriormente a ser desenvolvida com apoio de agentes de geração automática de código.

No papel de programador e analista, o aluno deverá definir claramente os requisitos da aplicação antes de começar a programar, especificando as funcionalidades pretendidas, os objetivos do sistema e as principais entidades envolvidas. O enunciado deverá ser suficientemente rigoroso para permitir uma implementação estruturada e deverá obrigar à organização do código em várias classes, promovendo uma abordagem modular e orientada a objetos.

A aplicação será desenvolvida para execução em ambiente de consola. Cada aluno tem liberdade para decidir o tema e o nível de complexidade do projeto, sendo esse fator considerado

posteriormente na avaliação. Os critérios específicos de avaliação serão definidos e comunicados pelo docente em momento oportuno.
Recomenda-se, no entanto, que o aluno comece por um enunciado simples, gerando uma primeira versão do código e dedicando tempo a compreender a estrutura e o funcionamento do código produzido. Só depois, de forma iterativa, deverá acrescentar novos requisitos e aumentar gradualmente a complexidade da aplicação, garantindo sempre que mantém uma boa organização e clareza no design do sistema.

És encorajado a utilizar todas as ferramentas de IA disponíveis para apoiar a tua aprendizagem, mas deves compreender totalmente o código gerado e o funcionamento do ambiente de desenvolvimento. Quando apresentares esta tarefa ao teu professor, serás questionado sobre estes aspetos.

Ferramenta de Apoio: Codex – OpenAI's Coding Agent

Para apoiar a implementação, podes utilizar o **Codex – OpenAI's coding agent**, um agente de programação com IA que te ajuda a escrever, rever e gerar código.

O Codex pode interagir com o teu repositório, **navegar pelo código existente, editar ficheiros e executar comandos no terminal conforme instruído**.
Podes aceder ao Codex de várias formas, incluindo:

- a partir do terminal com a **Codex CLI** (linha de comando);
- através da extensão para IDEs (como o VS Code), que oferece uma interface integrada para interagir com o agente (Recomendado);
- a partir da aplicação web ou dentro da tua conta do ChatGPT, dependendo do plano que tiveres.

O Codex não substitui o teu trabalho, mas funciona como um parceiro que responde a prompts e implementa código ou ajusta ficheiros com base nas tuas instruções, permitindo-te iterar mais rapidamente sobre o design da tua aplicação.

Como Utilizar o Codex de Forma Simples

Recomenda-se que adotes um processo iterativo e gradual:

1. Começa com um enunciado simples da aplicação.
2. Pede ao Codex para criar a **estrutura inicial do projeto**, por exemplo, novas classes como Tarefa, GestorTarefas, e uma classe Main.
3. Revê e compreende o código gerado antes de avançar para a etapa seguinte.
4. Acrescenta novas funcionalidades em etapas sucessivas, com prompts cada vez mais detalhados.

Exemplos de Prompts Simples para o Codex

- "Cria uma classe Tarefa em Java com atributos descricao e estado, e métodos para definir e obter esses valores."
- "Gera uma classe GestorTarefas que guarda uma lista de Tarefa e permite adicionar e listar tarefas."
- "Cria a classe Main que interage com o utilizador em consola para usar GestorTarefas."

Procedimento Resumido

1. Define os **requisitos da tua aplicação** (funcionalidades, entidades/classes, regras de negócio).
2. Configura o acesso ao **Codex** (CLI ou extensão no IDE).
3. Usa prompts de linguagem natural para pedir ao agente que gere/edite código em Java.
4. Revê, testa e compreende o código gerado.
5. Itera adicionando novas funcionalidades até a aplicação satisfazer os requisitos definidos.

Pontos Extra

1. **Testes e Documentação com IA**
   Utiliza o Codex para aprender o que são **testes unitários** e como funcionam. Pede ao agente para gerar e executar os testes unitários da tua aplicação.
   Para além disso, solicita ao Codex que crie os ficheiros de documentação do projeto, incluindo README.md, architecture.md e qualquer outra documentação relevante à aplicação que desenvolveste.
2. **Gestão de Código com GitHub**
   Cria uma conta no **GitHub** caso ainda não tenhas. Cria um repositório para o teu projeto e envia a aplicação para esse repositório (*push*).
   Aproveita a IA para aprender mais sobre **Git**, controlo de versões e boas práticas de gestão de código.

O código relativo a este trabalho deverá ser submetido até ao dia **15 de março**. A discussão do mesmo terá lugar nas aulas práticas até ao dia **29 de março de 2026**.

Bom trabalho, Jorge Branco, José Múrias, Andreia Artífice, Pedro Fazenda